



**VNiVERSiDAD
D SALAMANCA**

Facultad de Ciencias

Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

Desarrollo de una IA para jugar Riichi Mahjong

Development of an AI to Play Riichi Mahjong

Predicción y evaluación estratégica de descartes de Riichi Mahjong mediante Transformers

Prediction and Strategic Evaluation of Riichi Mahjong Discards Using Transformers

ANEXO V

Manual de Reproducibilidad

Autor

Julia Ruiperez de Brito

Tutor

Prof. Vidal Moreno Rodilla

Departamento

Informática y Automática

Salamanca

Junio 2026

Índice general

1	Introducción	2
2	Requisitos	2
3	Preparación de los Ficheros	2
4	Ejecución de la Evaluación	3
5	Resultados Esperados	4
6	Alcance y Limitaciones	4

1 Introducción

Este anexo describe el procedimiento necesario para reproducir la evaluación definitiva de los dos checkpoints presentada en la memoria. El objetivo es volver a generar las métricas predictivas, la clasificación estratégica, los ficheros CSV y las figuras a partir del código, la base de datos filtrada y los modelos conservados.

La unidad principal de reproducción es el notebook `11_independent_holdout_evaluation.ipynb`. Este cuaderno utiliza los últimos 52.563 estados de la tabla `Discard` de `strategic.db`, comenzando en el desplazamiento 500.000. El procedimiento no modifica los notebooks ni los informes anteriores.

La reproducción completa del entrenamiento original no tiene el mismo nivel de trazabilidad. En particular, del checkpoint 100k no se conservaron todos los metadatos necesarios para reconstruir íntegramente aquella ejecución. Por ello, este manual se centra en la reproducción de las predicciones y de los resultados finales a partir de los checkpoints disponibles.

2 Requisitos

La ejecución requiere Python 3.11 y las dependencias declaradas en `environment.yml`. Entre las bibliotecas principales se encuentran PyTorch, NumPy, Pandas, Matplotlib, Seaborn, Jupyter y tqdm. Puede utilizarse una GPU compatible con CUDA para reducir el tiempo de ejecución, aunque el notebook selecciona automáticamente la CPU cuando no existe una GPU disponible.

Desde la raíz del proyecto, el entorno puede crearse y activarse con:

```
conda env create -f environment.yml
conda activate riichi-ai
```

También deben estar disponibles los siguientes ficheros:

- `data/discard/strategic.db`: base de datos filtrada con 52.563 estados.
- `checkpoints/strategic_100k/strategic_baseline.pt`: checkpoint identificado como 100k.
- `checkpoints/strategic_500k/epoch_15.pt`: checkpoint final del entrenamiento con 500.000 estados.
- El directorio `src/`, que contiene la tokenización, la tensorización, el modelo y las métricas estratégicas.

Los datos originales proceden del dataset de Mahjong distribuido en Kaggle y derivado de registros de Tenhou. El fichero `strategic.db` utilizado aquí es la versión filtrada durante el proyecto, no la base original sin procesar.

3 Preparación de los Ficheros

Antes de ejecutar la evaluación debe comprobarse que el directorio de trabajo conserva la estructura esperada:

```
proyecto/  
|-- checkpoints/  
| |-- strategic_100k/strategic_baseline.pt  
| `-- strategic_500k/epoch_15.pt  
|-- data/discard/strategic.db  
|-- notebooks/11_independent_holdout_evaluation.ipynb  
|-- reports/  
|-- src/  
`-- environment.yml
```

El propio notebook realiza tres verificaciones sobre la base de datos: que la tabla contiene 552.563 filas, que sus identificadores forman una secuencia contigua entre 1 y 552.563 y que el conjunto reservado contiene exactamente 52.563 estados. Si alguna condición no se cumple, la ejecución se detiene mediante una aserción para evitar generar resultados a partir de una versión distinta de los datos.

Las rutas se calculan respecto de la raíz del proyecto. Por ello, el notebook puede abrirse desde el directorio notebooks/ o ejecutarse manteniendo la raíz del repositorio como directorio actual.

4 Ejecución de la Evaluación

La forma más sencilla de reproducir los resultados consiste en iniciar Jupyter Lab desde la raíz del proyecto:

```
jupyter lab
```

A continuación, debe abrirse notebooks/11_independent_holdout_evaluation.ipynb, seleccionar el entorno riichi-ai y ejecutar todas las celdas en orden. El cuaderno realiza las siguientes operaciones:

1. Comprueba la base de datos y registra la separación entre entrenamiento y evaluación.
2. Lee en orden los estados situados a partir del desplazamiento 500.000.
3. Aplica la misma normalización, tokenización y tensorización utilizada por los checkpoints.
4. Evalúa ambos modelos con `torch.inference_mode()` y calcula pérdida, Top-1, Top-3 y Top-5.
5. Calcula Shanten y Ukeire para los descartes comparados y asigna las categorías estratégicas.
6. Guarda las tablas y figuras definitivas sin sobrescribir los análisis anteriores.

La evaluación utiliza lotes de 64 estados y no mezcla las filas. No es necesario modificar las constantes del cuaderno para reproducir los resultados de la memoria.

5 Resultados Esperados

Al finalizar la ejecución deben aparecer los siguientes ficheros en `reports/holdout/`:

- `holdout_split.csv`
- `holdout_predictions.csv`
- `holdout_strategic_results.csv`
- `holdout_model_summary.csv`
- `holdout_category_distribution.csv`

Las figuras se guardan en `reports/figures/holdout/`. El resumen debe contener 52.563 estados evaluados por modelo y los siguientes valores, admitiendo únicamente diferencias mínimas de redondeo:

Cuadro 1: Métricas de referencia para comprobar la reproducción

Checkpoint	Pérdida	Top-1	Top-3	Top-5	Estratégica
100k	1,3533	55,81%	83,37%	91,25%	88,97%
500k	0,9440	65,64%	92,47%	97,71%	94,01%

Si los resultados difieren de forma sustancial, deben comprobarse primero las rutas, la versión de `strategic.db`, los checkpoints y el orden de ejecución de las celdas.

Como referencia de rendimiento, una ejecución con una GPU NVIDIA GeForce RTX 5060 Laptop y lotes de 64 muestras necesitó 167 segundos para obtener las predicciones de ambos checkpoints y 481 segundos para calcular Shanten, Ukeire y las categorías estratégicas. Los tiempos pueden variar según el hardware y la carga del sistema, pero permiten detectar ejecuciones anómalas y muestran que el cálculo heurístico es la etapa más costosa.

6 Alcance y Limitaciones

El procedimiento reproduce la evaluación final siempre que se disponga de los dos checkpoints y de la misma base de datos filtrada. No demuestra por sí mismo que los entrenamientos originales puedan repetirse de forma idéntica desde cero. El checkpoint 100k carece de un registro completo de época, optimizador y ejecución, mientras que el entrenamiento de 500.000 estados depende además del hardware y del comportamiento estocástico de PyTorch.

La separación del conjunto reservado se realiza por posición en la tabla y no por partida, porque el dataset procesado no contiene un identificador de partida. Se garantiza que las filas del conjunto final no se utilizaron para entrenar el modelo de 500.000 estados, pero no puede garantizarse que estados procedentes de una misma partida no aparezcan a ambos lados de la separación.

Por último, la precisión estratégica solo evalúa la eficiencia ofensiva representada mediante Shanten y Ukeire. No incorpora puntuación esperada, riesgo defensivo, situación completa de la partida ni valor posicional. Estas limitaciones deben mantenerse al interpretar una reproducción correcta de las métricas.